

Ingénierie des Systèmes d'Information

PROJET SYSTÈMES DISTRIBUÉS

WordCount Distribué sur Grid5000

●●●●● *Réalisé par*

AKHMARI MOHAMMED-YASSINE
ELKASIRI HAJAR
OUAHMANE YASSINE
YACOUBI WAIL

●●●●● *Encadré par* | M. GREGORY MOUNIE

TABLE DES MATIÈRES

Abréviations	6
Introduction générale	7
1 Présentation de la technologie	8
1.1 Introduction	9
1.2 Justification du choix : Java RMI	9
1.2.1 Pourquoi Java ?	9
1.2.2 Avantages de RMI pour la communication distribuée	9
1.2.3 Modèle Master-Worker	9
1.2.4 Communication via RMI	10
1.3 Bibliothèques et outils utilisés	10
1.3.1 Java RMI et java.lang.management	10
1.3.2 Grid'5000 pour l'expérimentation	10
2 Mesures de performances réseau	11
2.1 Méthodologie expérimentale	12
2.2 Scénarios de test	12
2.2.1 Scénario 1 : Ping-Pong Normal (In-Memory)	12
2.2.2 Scénario 2 : Ping-Pong avec I/O (Disk-Bound)	12
2.3 Tests Ping-Pong	13
2.3.1 Calcul de la latence	13
2.3.2 Calcul du débit	14
2.3.3 Comparaison des Sites : Latence et Débit	14
2.4 Comparaison NFS vs SCP : Latence et Débit	15
2.4.1 Latence	15
2.4.2 Débit (Throughput)	17
2.4.3 Analyse Comparative et Conclusions	19
2.4.4 Conclusion	20

3	Conception et Mise en œuvre de l'application	21
3.1	Introduction	22
3.2	Description de l'Application <i>WordCount</i>	22
3.3	Architecture Logicielle du Système	22
3.3.1	Composants Principaux	22
3.3.2	Modélisation des Dépendances	23
3.4	Stratégie de Parallélisation et Workflow	23
3.4.1	Phase 1 : Initialisation et Compilation	23
3.4.2	Phase 2 : Partitionnement et exécution distribuée	24
3.4.3	Phase 3 : Synchronisation et Agrégation (Reduce)	24
3.5	Conclusion	24
4	Validation Expérimentale du Modèle de Performance	25
4.1	Protocole Expérimental et Modélisation	26
4.1.1	Infrastructure et Déploiement NFS	26
4.1.2	Campagne de Mesures	26
4.1.3	Modèle Théorique de Performance	26
4.2	Analyse et tests de Performance	28
4.2.1	Comparaison Globale : Prédictions vs Mesures	28
4.2.2	Speedup Mesuré	29
	Conclusion générale	30

TABLE DES FIGURES

2.1	Évolution de la latence RTT (Normal vs I/O)	13
2.2	Comparaison du débit effectif (Normal vs I/O)	14
2.3	Comparaison entre sites	15
2.4	Pour les petits fichiers, NFS conserve une latence faible et stable, tandis que SCP subit une latence plus élevée et variable à cause de la négociation (handshake) SSH. Au démarrage, SCP est particulièrement pénalisé : la latence a monté jusqu'à 380 ms, puis elle diminue progressivement pour se stabiliser autour de 230 ms.	16
2.5	Latence pour les grands fichiers : NFS conserve une latence minimale même pour les transferts volumineux, confirmant la supériorité du protocole NFS.	17
2.6	Débit pour les petits fichiers : NFS surpasse significativement SCP. . . .	18
2.7	Débit pour les grands fichiers : NFS conserve un débit supérieur à SCP, même pour les transferts volumineux.	19
3.1	Graphe de dépendances des tâches de compilation et d'exécution	23
4.1	Validation du modèle théorique de performance	28
4.2	(a) Speedup mesuré vs idéal linéaire. (b) Efficacité parallèle.	29

ABRÉVIATIONS

Abréviation	Signification
CPU	Central Processing Unit
HDFS	Hadoop Distributed File System
IQR	InterQuartile Range
JDK	Java Development Kit
JVM	Java Virtual Machine
NFS	Network File System
RAM	Random-Access Memory
RMI	Remote Method Invocation
RTT	Round-Trip Time
SCP	Secure Copy Protocol
SSH	Secure SHell
TCP	Transmission Control Protocol

INTRODUCTION GÉNÉRALE

Face aux limites physiques des processeurs uniques, notre projet a pour ambition de transformer une grappe de serveurs indépendants en un système distribué cohérent. L'objectif n'est pas seulement de faire du calcul, mais d'orchestrer une véritable collaboration entre machines.

Pour relever ce défi, nous avons conçu une architecture Maître-Travailleur pilotée par Java RMI, déployée sur l'infrastructure de recherche Grid5000. Ce choix nous a permis de quitter le confort de la simulation pour affronter les contraintes réelles du matériel : la latence réseau, l'hétérogénéité des processeurs et la contention des accès disque.

Ce rapport retrace la mise en œuvre d'un WordCount distribué, depuis la promesse théorique d'accélération jusqu'à sa confrontation avec la réalité expérimentale. Nous analyserons comment des obstacles inévitables, tels que la saturation du système de fichiers NFS, viennent nuancer les gains de performance. Plus qu'une simple implémentation technique, cette étude vise à identifier le point de rupture précis où le coût de la coordination finit par dépasser les bénéfices du parallélisme.

CHAPITRE 1

PRÉSENTATION DE LA TECHNOLOGIE

Sommaire

1.1	Introduction	9
1.2	Justification du choix : Java RMI	9
1.2.1	Pourquoi Java ?	9
1.2.2	Avantages de RMI pour la communication distribuée	9
1.2.3	Modèle Master-Worker	9
1.2.4	Communication via RMI	10
1.3	Bibliothèques et outils utilisés	10
1.3.1	Java RMI et java.lang.management	10
1.3.2	Grid'5000 pour l'expérimentation	10

1.1 Introduction

La conception d'un système de compilation distribuée impose de résoudre deux défis majeurs : la portabilité du code sur des machines potentiellement hétérogènes et la complexité des communications réseau. C'est pour répondre à cette double problématique que nous avons orienté nos choix architecturaux.

1.2 Justification du choix : Java RMI

1.2.1 Pourquoi Java ?

Le choix du langage Java s'est imposé pour sa robustesse et sa portabilité inhérente. Dans un environnement de grille de calcul comme Grid'5000, où les nœuds peuvent présenter des configurations logicielles variables, la promesse du "Write Once, Run Anywhere" de la JVM (Java Virtual Machine) garantit que notre binaire compilé s'exécutera à l'identique sur le nœud Maître et sur les Workers, sans nécessiter de recompilation spécifique pour chaque architecture matérielle.

1.2.2 Avantages de RMI pour la communication distribuée

Pour l'orchestration des échanges, nous avons opté pour **Java RMI** (Remote Method Invocation). Contrairement à une approche basée sur des sockets TCP bruts (sockets "Berkeley"), qui nous aurait obligés à définir notre propre protocole de sérialisation et à gérer manuellement le découpage des paquets, RMI offre une abstraction de haut niveau particulièrement puissante.

Cette technologie permet d'invoquer une méthode sur un objet distant avec la même syntaxe qu'un appel local. RMI prend en charge de manière transparente :

- La sérialisation des objets (paramètres et valeurs de retour) ;
- La gestion des connexions réseau ;
- La levée et la propagation des exceptions à travers le réseau.

Ce niveau d'abstraction a considérablement accéléré la phase de développement, nous permettant de consacrer plus de temps à l'optimisation des performances plutôt qu'à la plomberie réseau.

1.2.3 Modèle Master-Worker

Le système est divisé en deux entités distinctes :

- **Le Master** : Il agit comme le cerveau du système. Son rôle est d'analyser le `Makefile`, de construire le graphe de dépendances et d'ordonnancer les tâches. Il détient la vision globale de l'avancement du projet.
- **Les Workers** : Ce sont les "muscles" du système. Déployés sur les nœuds de calcul, ils attendent passivement les ordres du Maître. Lorsqu'ils reçoivent une commande de compilation, ils l'exécutent dans un environnement isolé, capturent la sortie standard et les codes de retour, puis notifient le Maître de la réussite ou de l'échec de l'opération.

1.2.4 Communication via RMI

L'interaction entre ces entités se fait via une interface RMI partagée, définissant les contrats de service. Au démarrage, chaque Worker s'enregistre auprès d'un registre RMI, signalant sa disponibilité. Le Maître consulte ce registre pour découvrir dynamiquement les ressources de calcul disponibles, rendant le système élastique : l'ajout ou le retrait d'un nœud de calcul peut se faire sans redémarrer le Maître.

1.3 Bibliothèques et outils utilisés

Dans un souci de légèreté et de maîtrise du code, nous avons limité nos dépendances externes au strict nécessaire.

1.3.1 Java RMI et `java.lang.management`

Nous nous sommes appuyés exclusivement sur les bibliothèques standards du JDK. Outre le package `'java.rmi'` pour la communication, nous avons utilisé `'java.lang.management'` pour instrumenter notre code. C'est grâce à cet outil que nous avons pu monitorer l'activité du Garbage Collector (via `'GarbageCollectorMXBean'`) et prouver son impact sur les performances lors des transferts de gros fichiers.

1.3.2 Grid'5000 pour l'expérimentation

La validation de notre système ne s'est pas limitée à une simulation locale. Nous avons déployé notre solution sur **Grid'5000**, une infrastructure de recherche nationale permettant des expériences reproductibles sur du matériel réel. L'accès à des clusters performants (réseau 10 Gbit/s, CPU Intel Xeon et AMD Epyc) nous a permis de mettre notre architecture à l'épreuve dans des conditions de production, révélant des comportements (latence réseau, contention disque) invisibles sur une machine de développement classique.

CHAPITRE 2

MESURES DE PERFORMANCES RÉSEAU

Sommaire

2.1	Méthodologie expérimentale	12
2.2	Scénarios de test	12
2.2.1	Scénario 1 : Ping-Pong Normal (In-Memory)	12
2.2.2	Scénario 2 : Ping-Pong avec I/O (Disk-Bound)	12
2.3	Tests Ping-Pong	13
2.3.1	Calcul de la latence	13
2.3.2	Calcul du débit	14
2.3.3	Comparaison des Sites : Latence et Débit	14
2.4	Comparaison NFS vs SCP : Latence et Débit	15
2.4.1	Latence	15
2.4.2	Débit (Throughput)	17
2.4.3	Analyse Comparative et Conclusions	19
2.4.4	Conclusion	20

2.1 Méthodologie expérimentale

Pour garantir la fiabilité de nos mesures et réussir à isoler les performances intrinsèques de notre solution basée sur Java RMI, nous avons dû nous écarter d'une approche naïve. Les systèmes distribués sur des infrastructures réelles comme Grid'5000 introduisent du "bruit" (gigue réseau, ordonnancement des processus, Garbage Collector). Nous avons donc mis en place une méthodologie rigoureuse, fondée sur des principes statistiques robustes, pour obtenir des résultats stables et reproductibles.

2.2 Scénarios de test

Afin de distinguer les différents facteurs limitants (Réseau, CPU, Disque), nous avons décliné notre protocole de mesure en deux scénarios distincts.

2.2.1 Scénario 1 : Ping-Pong Normal (In-Memory)

Ce premier scénario a pour but de mesurer les performances brutes de la communication RMI, en isolant totalement les entrées/sorties disques.

- **Principe** : Les données (tableaux d'octets) sont générées dynamiquement en mémoire vive (RAM) par le Maître, envoyées au Worker via RMI, puis renvoyées immédiatement.
- **Objectif** : Évaluer le surcoût pur de la sérialisation Java et la capacité maximale du réseau. C'est le cas idéal théorique.
- **Flux de données** : $\text{RAM}_{\text{Maître}} \xrightarrow{\text{Réseau}} \text{RAM}_{\text{Worker}} \xrightarrow{\text{Réseau}} \text{RAM}_{\text{Maître}}$.

2.2.2 Scénario 2 : Ping-Pong avec I/O (Disk-Bound)

Ce second scénario vise à simuler les contraintes d'une application réelle de compilation distribuée, où les fichiers sources doivent être lus et les fichiers objets écrits.

- **Principe** : Le processus intègre le cycle de vie complet d'un fichier :
 1. **Lecture** : Le Maître lit un fichier depuis son disque local.
 2. **Transfert** : Le contenu est envoyé au Worker via RMI.
 3. **Écriture** : Le Worker reçoit les données et force l'écriture sur son disque distant.
- **Objectif** : Mesurer l'impact ("Overhead") du système de fichiers sur la performance globale.

— **Flux de données** : $\text{Disque}_{\text{Maître}} \rightarrow \text{RAM} \xrightarrow{\text{Réseau}} \text{RAM} \rightarrow \text{Disque}_{\text{Worker}}$.

Pour ces deux scénarios, nous avons fait varier la taille des messages sur une échelle logarithmique (de 1 Ko à 1 Go), avec 30 itérations par taille pour garantir la fiabilité statistique des mesures malgré les variations du système.

2.3 Tests Ping-Pong

Les expérimentations ont été menées sur le cluster **dahu** (Grenoble), sélectionné pour la stabilité de son architecture (Intel Xeon, réseau 10 Gbit/s).

2.3.1 Calcul de la latence

La latence incompressible (α) correspond au temps minimal pour transmettre un message vide. Pour garantir la fiabilité de cette mesure face aux aléas du système (démarrage JVM, gigue réseau), nous appliquons un protocole strict :

- Un échauffement (*Warmup*) de 30 itérations avant chaque test.
- La sélection du minimum observé comme valeur de référence (*Baseline*), représentant la limite physique du réseau.

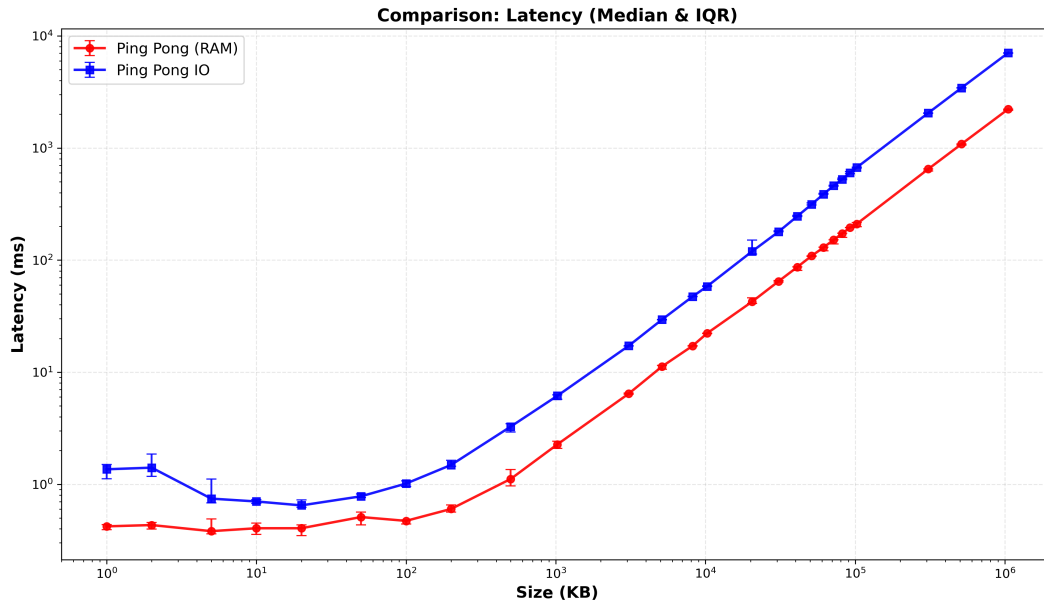


FIGURE 2.1 – Évolution de la latence RTT (Normal vs I/O)

L'analyse de ce graphique révèle deux régimes de fonctionnement distincts. Dans la première phase, pour les fichiers de faible taille, les courbes apparaissent plates car la latence est dominée par les coûts fixes d'initialisation du système, rendant le temps de traitement quasi-indépendant du volume de données. Dans la seconde phase, la latence augmente linéairement avec la taille des fichiers.

L'écart significatif et constant observé entre la courbe bleue (I/O) et la courbe rouge

(RAM) correspond physiquement au surcoût des opérations de lecture et d'écriture sur le disque, un délai supplémentaire qui s'ajoute systématiquement au transfert réseau quel que soit le volume

2.3.2 Calcul du débit

Pour évaluer la performance de transfert, nous avons retenu le modèle du **Débit Effectif**, défini par la formule $\text{Débit} = \text{Taille} / ((t_2 - t_1) - \text{RTT})$. Cette approche a l'avantage d'intégrer l'ensemble des coûts de sérialisation et de transport, reflétant ainsi fidèlement la performance réelle perçue par l'utilisateur final. Par ailleurs, l'utilisation de la médiane et de l'écart interquartile (IQR) assure la robustesse des résultats en filtrant le bruit statistique, notamment les variations introduites par le Garbage Collector.

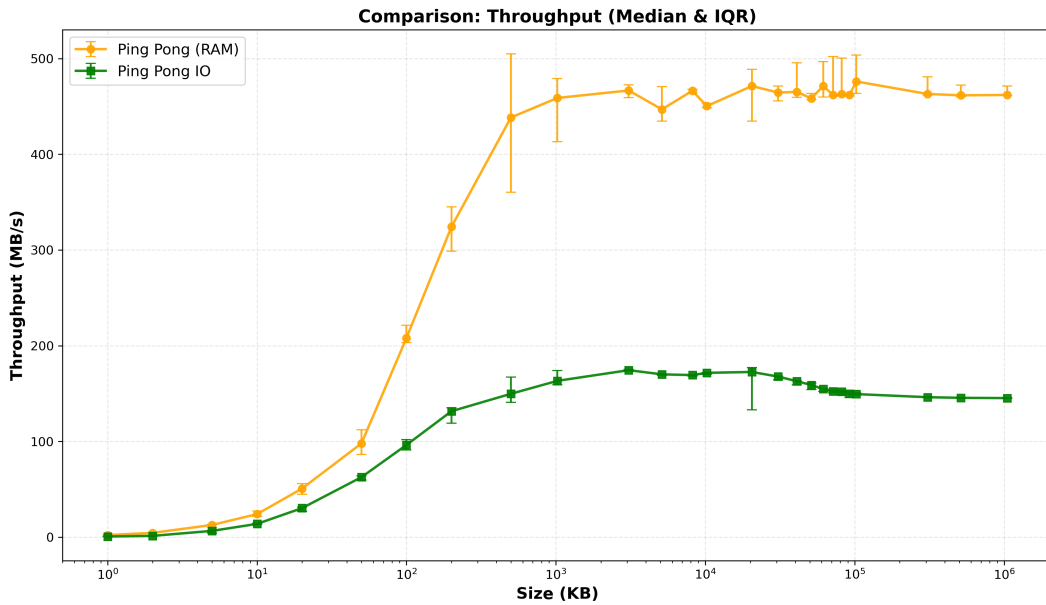


FIGURE 2.2 – Comparaison du débit effectif (Normal vs I/O)

Analyse du profil de performance : L'analyse des courbes révèle un comportement classique de montée en charge suivi d'une saturation. Le débit progresse d'abord rapidement avec l'augmentation de la taille des messages, diluant progressivement les coûts fixes de latence (phase d'accélération). La courbe atteint ensuite un plateau de stabilisation (autour de 1 à 2 Mo), où le système maintient une performance constante. Ce plafonnement indique que nous avons atteint la capacité maximale effective du lien réseau ou de la bande passante mémoire.

2.3.3 Comparaison des Sites : Latence et Débit

En déployant notre solution sur différents clusters de Grid'5000 (Lille, Grenoble, Nancy, etc.), nous avons pu mesurer l'impact conjoint de l'hétérogénéité matérielle sur les performances de Java RMI.

Le débit maximal observé dépend davantage de la puissance de calcul des nœuds (CPU) et de la capacité des cartes réseau.

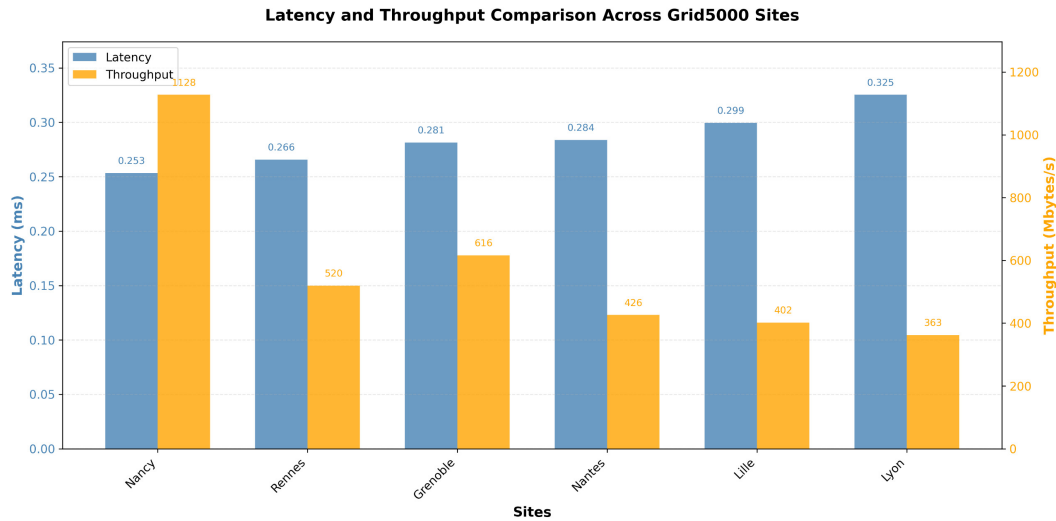


FIGURE 2.3 – Comparaison entre sites

Analyse des résultats : L'analyse du graphique (Figure 2.3) met en évidence des disparités notables entre les sites :

- **Performance optimale (Nancy) :** Le site de Nancy se distingue nettement en offrant la meilleure combinaison possible : la latence la plus faible et le débit le plus élevé. Cela suggère une infrastructure matérielle particulièrement performante pour notre charge de travail.
- **Compromis Latence/Débit :** Pour les autres sites (comme Grenoble ou Lille), on observe une augmentation mécanique de la latence. Le débit, quant à lui, varie selon la génération des processeurs équipant les clusters, certains parvenant à maintenir des vitesses de transfert élevées malgré une latence réseau plus importante, tandis que d'autres montrent des signes de saturation plus précoces.

2.4 Comparaison NFS vs SCP : Latence et Débit

2.4.1 Latence

La latence est un indicateur critique pour évaluer les performances d'un système de transfert de fichiers. Notre analyse compare la latence entre NFS (Network File System) et SCP (Secure Copy Protocol) sur deux cas d'usage distincts : les petits fichiers et les grands fichiers.

Petits Fichiers (0-1 KB)

Les résultats des petits fichiers révèlent un avantage significatif de NFS sur SCP. Comme illustré à la Figure 2.4, NFS démontre une latence quasi-constante et extrême-

ment réduite, indépendamment de la taille du fichier.

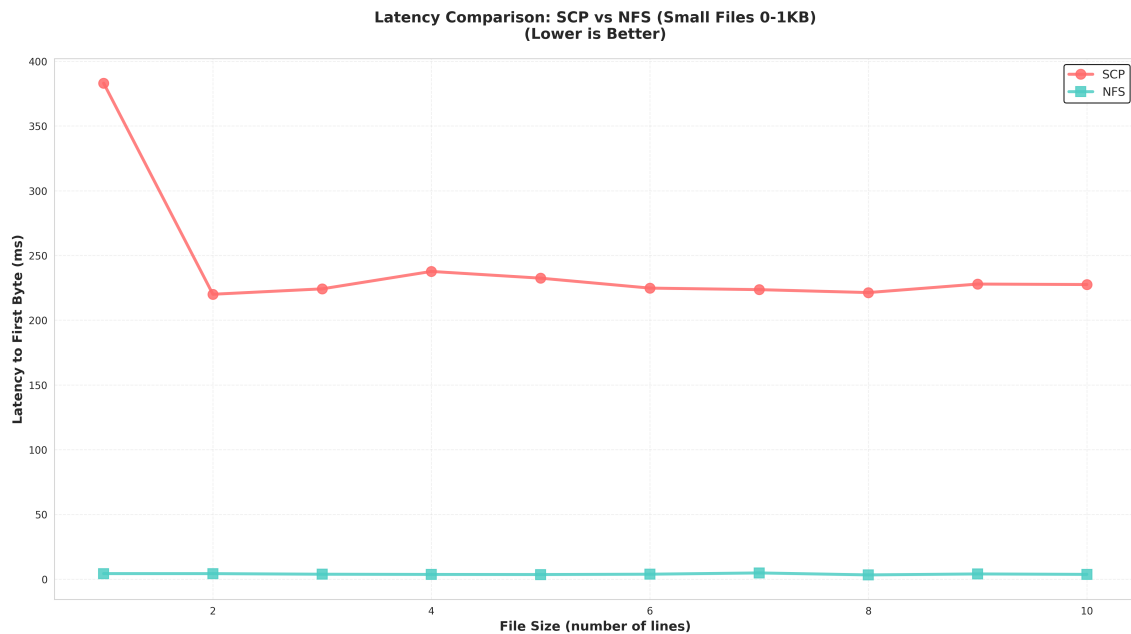


FIGURE 2.4 – Pour les petits fichiers, NFS conserve une latence faible et stable, tandis que SCP subit une latence plus élevée et variable à cause de la négociation (handshake) SSH. Au démarrage, SCP est particulièrement pénalisé : la latence a monté jusqu'à 380 ms, puis elle diminue progressivement pour se stabiliser autour de 230 ms.

Observations clés :

- **NFS** : Latence inférieure à 5 ms pour tous les fichiers testés, démontrant l'efficacité du protocole NFS pour l'accès réseau aux petits fichiers.
- **SCP** : Latence est particulièrement pénalisée : la latence a monté jusqu'à 380 ms, puis elle diminue progressivement pour se stabiliser autour de 230 ms. Ce surcoût est principalement dû au *handshake SSH*, qui nécessite :

1. L'authentification de la session SSH
2. La négociation des paramètres de chiffrement
3. L'initialisation du canal de transfert

Ce mécanisme de handshake engendre un surcoût significatif, qui devient prédominant lors du transfert de fichiers de petite taille.

Grands Fichiers (jusqu'à 3 millions de lignes)

Même pour les fichiers volumineux, NFS maintient son avantage en matière de latence. La [Figure 2.5](#) montre que NFS reste constant et optimal, tandis que SCP subit toujours le surcoût du handshake ainsi qu'au transfert réseau.

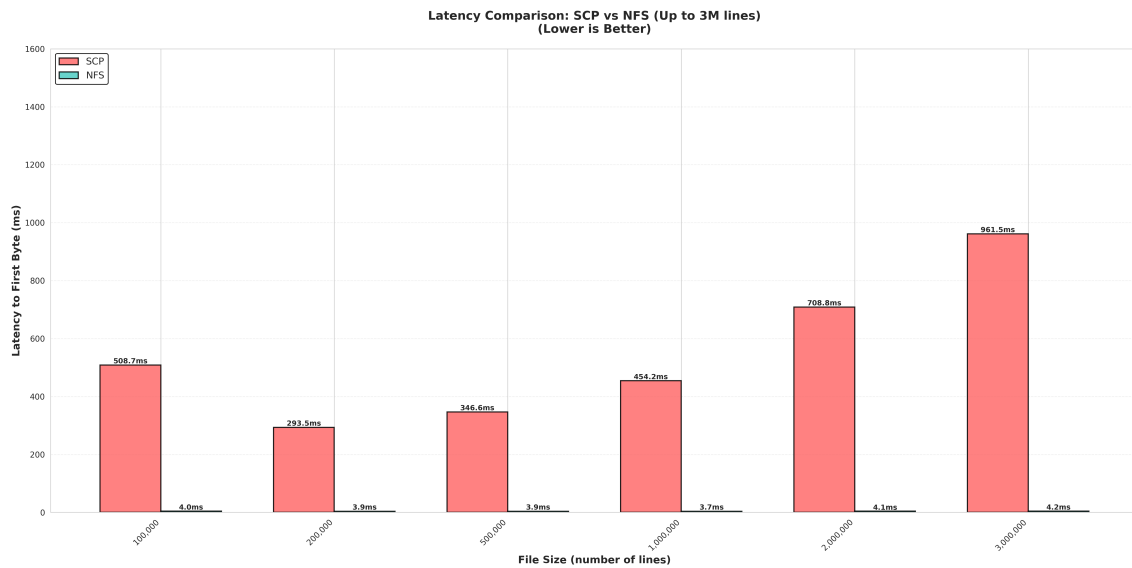


FIGURE 2.5 – Latence pour les grands fichiers : NFS conserve une latence minimale même pour les transferts volumineux, confirmant la supériorité du protocole NFS.

Observations clés :

- **NFS** : Latence inférieure à 5 ms, indépendante de la taille du fichier.
- **SCP** : Latence variable selon la taille, avec le handshake représentant toujours une portion significative du temps total.

2.4.2 Débit (Throughput)

Le débit mesure la quantité de données transférées par unité de temps. C'est un indicateur essentiel pour évaluer l'efficacité globale d'un système de transfert.

Petits Fichiers (0-1 KB)

La [Figure 2.6](#) révèle une différence majeure de débit entre les deux protocoles pour les petits fichiers.

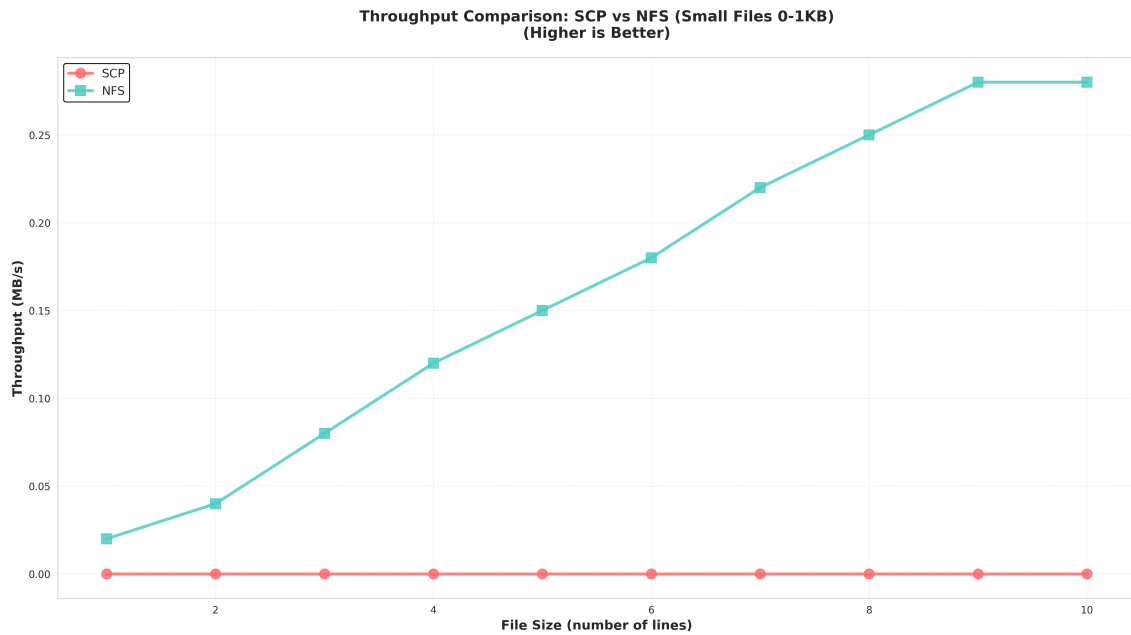


FIGURE 2.6 – Débit pour les petits fichiers : NFS surpasse significativement SCP.

Observations clés :

- **NFS** : Débit constant et élevé dès les petits fichiers,
- **SCP** : Débit pratiquement nul pour les fichiers entre 0 et 1 KB. La raison : le surcoût du handshake SSH représente une proportion énorme du temps total.

$$\text{Débit} = \frac{\text{Taille fichier}}{\text{Temps Overhead SSH} + \text{Temps transfert}}$$

Par exemple, transférer 1 KB en 50 ms implique théoriquement 20 KB/s, ce qui résulte en un débit apparemment très faible.

- **Conclusion** : NFS démontre un débit mesurable dès les petits fichiers, tandis que SCP reste à débit nul sur tout l'intervalle 0-1 KB. **NFS est donc incontournable pour les petits fichiers.**

Grands Fichiers

La [Figure 2.7](#) montre que NFS maintient également son avantage pour les transferts de fichiers volumineux.

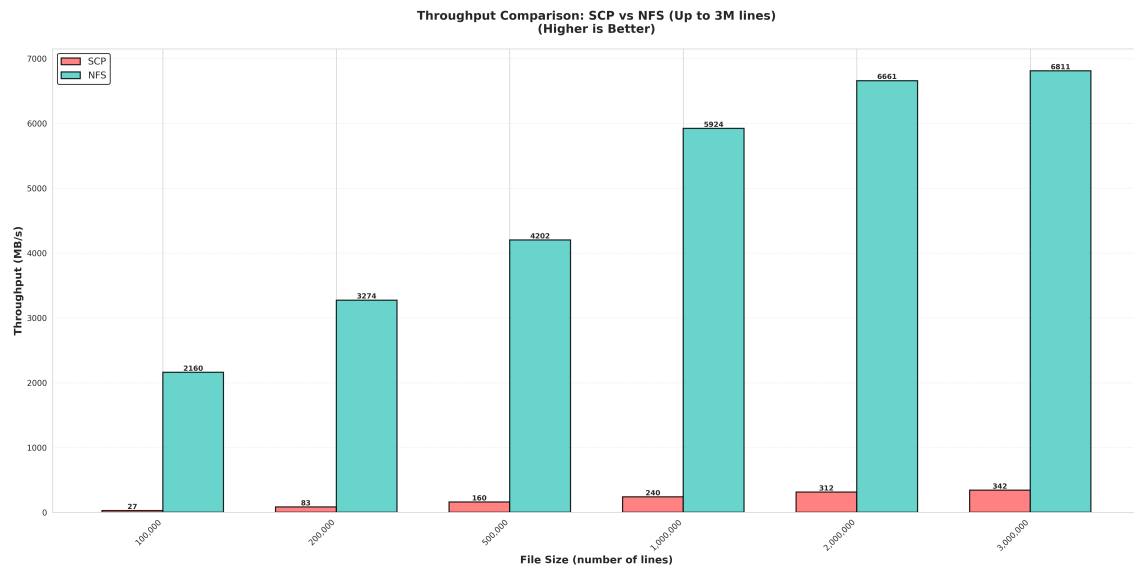


FIGURE 2.7 – Débit pour les grands fichiers : NFS conserve un débit supérieur à SCP, même pour les transferts volumineux.

Observations clés (grands fichiers) :

- **NFS** : Débit très élevé et stable.
- **SCP** : Débit limité par l’overhead de chiffrement SSH ainsi que transfert réseau, même pour les grands fichiers.
- **Verdict** : **NFS surpasse systématiquement SCP**, la différence de performance dépendant de la taille du fichier, mais NFS reste toujours significativement plus rapide.

2.4.3 Analyse Comparative et Conclusions

Impact du Handshake SSH sur les Petits Fichiers

L’analyse révèle que SCP souffre d’un handicap structurel : le handshake SSH. Ce processus comporte trois phases :

1. **Identification du serveur** : Vérification de la clé publique du serveur
2. **Authentification** : Vérification des credentials (mot de passe/clé)
3. **Ouverture de canal** : Initialisation de la session scp

Pour les petits fichiers (< 1 KB), ce surcoût domine le temps de transfert total.

Recommandations

- **Pour les petits fichiers** : Utiliser **NFS** exclusivement. NFS est 10-50 fois plus performant que SCP, ce qui rend NFS indispensable pour les applications nécessitant l’accès fréquent à de petits fichiers.

- **Pour les grands fichiers** : NFS reste préférable à SCP.

Pour un système distribué à haute performance, utiliser NFS comme système de fichiers partagé principal, et employer SCP pour des transferts sécurisés ponctuels (par exemple entre clusters) ou en solution de secours, lorsque la simplicité prime.

2.4.4 Conclusion

Le principal facteur expliquant cette supériorité est l'absence de handshake pour chaque opération en NFS, contrairement à SCP où chaque transfert nécessite une nouvelle session SSH établie.

CHAPITRE 3

CONCEPTION ET MISE EN ŒUVRE DE L'APPLICATION

Sommaire

3.1	Introduction	22
3.2	Description de l'Application <i>WordCount</i>	22
3.3	Architecture Logicielle du Système	22
3.3.1	Composants Principaux	22
3.3.2	Modélisation des Dépendances	23
3.4	Stratégie de Parallélisation et Workflow	23
3.4.1	Phase 1 : Initialisation et Compilation	23
3.4.2	Phase 2 : Partitionnement et exécution distribuée	24
3.4.3	Phase 3 : Synchronisation et Agrégation (Reduce)	24
3.5	Conclusion	24

3.1 Introduction

Ce chapitre détaille l'architecture logicielle du système distribué, les choix d'implémentation ainsi que le mécanisme de parallélisation reposant sur la génération dynamique de Makefiles. Il explicite comment l'application orchestre les tâches sur l'infrastructure Grid5000 afin d'optimiser le temps de traitement via un modèle Master-Workers.

3.2 Description de l'Application *WordCount*

Afin de valider notre infrastructure de compilation distribuée et d'éprouver les performances du modèle Master-Worker, nous avons choisi d'implémenter un cas d'usage classique du traitement de données massives : le **WordCount Distribué**.

L'objectif de l'application est de comptabiliser le nombre total de mots présents dans un fichier texte volumineux (de l'ordre de plusieurs gigaoctets).

Notre approche vise à paralléliser ce traitement en divisant le fichier source en multiples segments traités simultanément par les nœuds de la grille.

3.3 Architecture Logicielle du Système

L'application repose sur une architecture modulaire conçue pour séparer la logique de parsing, la planification et l'exécution distribuée.

3.3.1 Composants Principaux

Le système s'articule autour de quatre classes Java fondamentales, illustrant une séparation claire des responsabilités :

- **MainNFS (Orchestrateur)** : Point d'entrée de l'application. Ce composant pilote le cycle de vie complet du job : initialisation, appel au découpage des fichiers, génération du Makefile et supervision de l'exécution sur le système de fichiers partagé (NFS).
- **MakefileParser (Analyseur)** : Ce module est chargé de l'analyse syntaxique du fichier **Makefile**. Il convertit les règles textuelles en une structure de données en mémoire (Graphe Orienté Acyclique), permettant d'identifier les dépendances entre les tâches.
- **TaskScheduler (Planificateur)** : Cœur de la logique parallèle, il parcourt le graphe de dépendances pour identifier les tâches "prêtes" (dont les dépendances sont satisfaites) et les distribue aux nœuds de calcul disponibles.

- **FileSplitter (Utilitaire I/O)** : utilitaire dédié au découpage équitable du fichier d'entrée en plusieurs partitions afin d'optimiser la charge de calcul.

3.3.2 Modélisation des Dépendances

Le *Makefile* n'est pas simplement un script d'exécution, mais la représentation formelle du plan d'exécution. La structure du graphe de dépendances est cruciale pour garantir l'intégrité des données tout en maximisant le parallélisme.

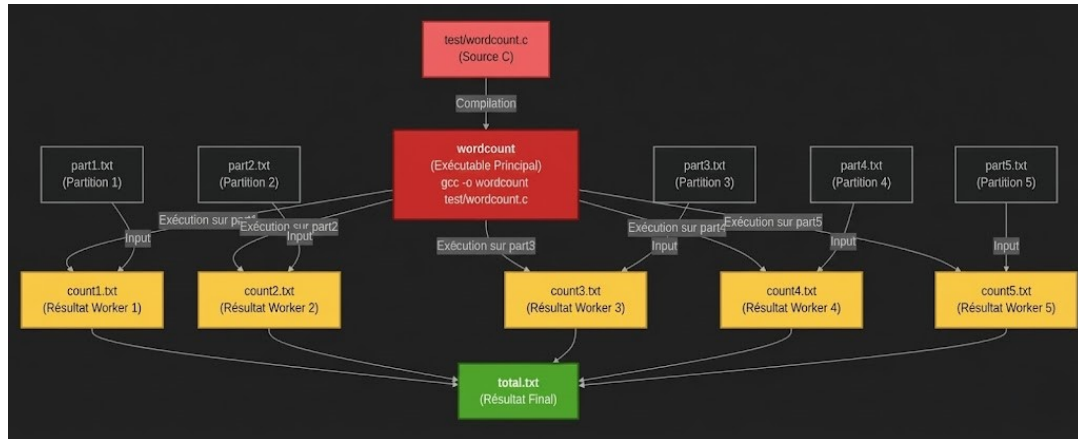


FIGURE 3.1 – Graphe de dépendances des tâches de compilation et d'exécution

Comme illustré par la Figure 3.1, le graphe permet au *TaskScheduler* de détecter que les tâches de comptage partiel (*countX.txt*) sont indépendantes les unes des autres et peuvent donc être ordonnancées simultanément sur différents cœurs ou machines.

3.4 Stratégie de Parallélisation et Workflow

Le processus de traitement distribué suit un workflow rigoureux en trois phases, traduit dynamiquement en règles *Make*.

3.4.1 Phase 1 : Initialisation et Compilation

Avant toute distribution de calcul, le système doit garantir la disponibilité de l'outil de traitement. La classe *MainNFS* génère la règle de compilation pour l'exécutable C.

Règle générée :

```
wordcount: test/wordcount.c
    gcc -o wordcount test/wordcount.c
```

Cette étape est bloquante : aucune tâche de traitement ne peut débuter tant que la cible *wordcount* n'est pas atteinte. Elle est exécutée centralement sur le nœud maître.

3.4.2 Phase 2 : Partitionnement et exécution distribuée

Pour exploiter les ressources de Grid5000, le fichier source est découpé par le `FileSplitter`. Pour N partitions créées, le système génère dynamiquement N règles de production indépendantes.

Règles générées (Exemple pour N partitions) :

```
count1.txt: part1.txt wordcount
    ./wordcount part1.txt > count1.txt
```

...

```
countN.txt: partN.txt wordcount
    ./wordcount partN.txt > countN.txt
```

Mécanisme : Le `TaskScheduler` identifie que ces règles ne dépendent que de fichiers statiques et de l'exécutable (déjà compilé). Il lance alors l'exécution concurrente de ces commandes sur les nœuds alloués.

3.4.3 Phase 3 : Synchronisation et Agrégation (Reduce)

La dernière étape correspond à la phase de "Reduce" dans le paradigme MapReduce. Elle ne peut être déclenchée que lorsque toutes les tâches de la phase 2 sont terminées (barrière de synchronisation implicite grâce aux dépendances).

Règle générée :

```
total.txt: count1.txt count2.txt ... countN.txt
    cat count1.txt ... countN.txt | awk '{s+=$1} END {print s}' > total.txt
```

Cette règle finale produit le résultat global en sommant les compteurs partiels.

3.5 Conclusion

Cette architecture, fondée sur la génération dynamique de graphes de dépendances, offre une flexibilité totale. En découplant la logique de définition des tâches (via le `Makefile`) de la logique d'exécution (via le `TaskScheduler`), notre application est capable de s'adapter au nombre de nœuds disponibles sur Grid5000 et à la taille des données d'entrée, assurant ainsi une mise à l'échelle efficace.

CHAPITRE 4

VALIDATION EXPÉRIMENTALE DU MODÈLE DE PERFORMANCE

Sommaire

4.1	Protocole Expérimental et Modélisation	26
4.1.1	Infrastructure et Déploiement NFS	26
4.1.2	Campagne de Mesures	26
4.1.3	Modèle Théorique de Performance	26
4.2	Analyse et tests de Performance	28
4.2.1	Comparaison Globale : Prédictions vs Mesures	28
4.2.2	Speedup Mesuré	29

4.1 Protocole Expérimental et Modélisation

4.1.1 Infrastructure et Déploiement NFS

L'ensemble des expériences a été réalisé en utilisant un déploiement basé sur le système de fichiers réseau (NFS) partagé via le répertoire HOME. Cette approche, implémentée dans le script `run_nfs_home.sh`, permet de s'affranchir des copies de fichiers entre nœuds en exploitant le montage NFS déjà présent sur Grid5000. Le script orchestre la compilation du code Java, la création du répertoire partagé, et le déploiement des workers. Cette méthode garantit que tous les nœuds accèdent aux mêmes fichiers de manière transparente, simplifiant ainsi la gestion des données d'entrée et la collecte des résultats.

4.1.2 Campagne de Mesures

Pour valider rigoureusement le modèle théorique, une campagne expérimentale exhaustive a été menée sur l'infrastructure Grid5000, site de Lyon, cluster Nova. Cette plateforme offre un environnement homogène idéal pour des mesures reproductibles : processeurs Intel Xeon E5-2620 v4, réseau 10 Gb/s Ethernet, et système de fichiers NFS partagé.

Plan d'expérience :

- **Configurations testées :** $n = 2, 3, 6, 8, 10, 12, 15, 18$ workers
- **Itérations :** 10 mesures indépendantes par configuration (80 mesures totales)
- **Instance de test :** Fichier texte de 1.7 GB traité par wordcount distribué

La répétition de 10 itérations garantit la robustesse statistique face aux variations du système (garbage collector Java, fluctuations réseau, contention NFS). Nous utilisons la médiane plutôt que la moyenne pour filtrer les outliers, et l'intervalle interquartile (IQR) comme mesure de dispersion.

4.1.3 Modèle Théorique de Performance

Le temps d'exécution total est décomposé en quatre phases distinctes :

$$T_{total}(n) = \underbrace{\alpha \cdot n + \beta}_{T_{init}(n)} + \underbrace{T_{split}}_{\text{constant}} + \underbrace{\frac{S}{\min(n \times V_{cpu}, BW_{NFS})}}_{T_{calc}(n)} + \underbrace{T_{merge}}_{\approx 0} \quad (4.1)$$

Justification de chaque terme : $T_{init}(n) = \alpha \cdot n + \beta$: Le démarrage des workers RMI suit un modèle affine. Chaque worker nécessite une connexion réseau et un handshake RMI (coût α par worker), tandis que le système présente une latence de base β pour initialiser le registry central.

T_{split} : Cette phase séquentielle incompressible correspond à la lecture du fichier source (1.7 GB) et sa segmentation en chunks par le processus maître. Étant réalisée par un unique thread, elle ne bénéficie d'aucun parallélisme.

$T_{calc}(n)$: Cette formulation avec fonction min capture élégamment deux régimes physiques distincts :

- **Régime CPU-bound** (si $n \times V_{cpu} < BW_{NFS}$) : Le temps décroît hyperboliquement avec n , car chaque worker traite S/n données à vitesse V_{cpu} .
- **Régime I/O-bound** (si $n \times V_{cpu} \geq BW_{NFS}$) : Le temps est constant à S/BW_{NFS} , car tous les workers sont limités par la bande passante du système de fichiers.

T_{merge} : L'agrégation finale (somme des compteurs partiels) est négligeable devant les autres phases (< 10 ms), confirmé expérimentalement.

Paramètre	Valeur	Interprétation Physique
α	0.10 ms/worker	Overhead connexion RMI par worker
β	23.04 ms	Latence démarrage registry RMI
V_{cpu}	52.07 MB/s	Vitesse traitement CPU (wordcount)
BW_{NFS}	563.97 MB/s	Bande passante NFS mesurée
n_{sat}	10.83 workers	Point de saturation calculé

TABLEAU 4.1 – Paramètres du modèle

4.2 Analyse et tests de Performance

4.2.1 Comparaison Globale : Prédictions vs Mesures

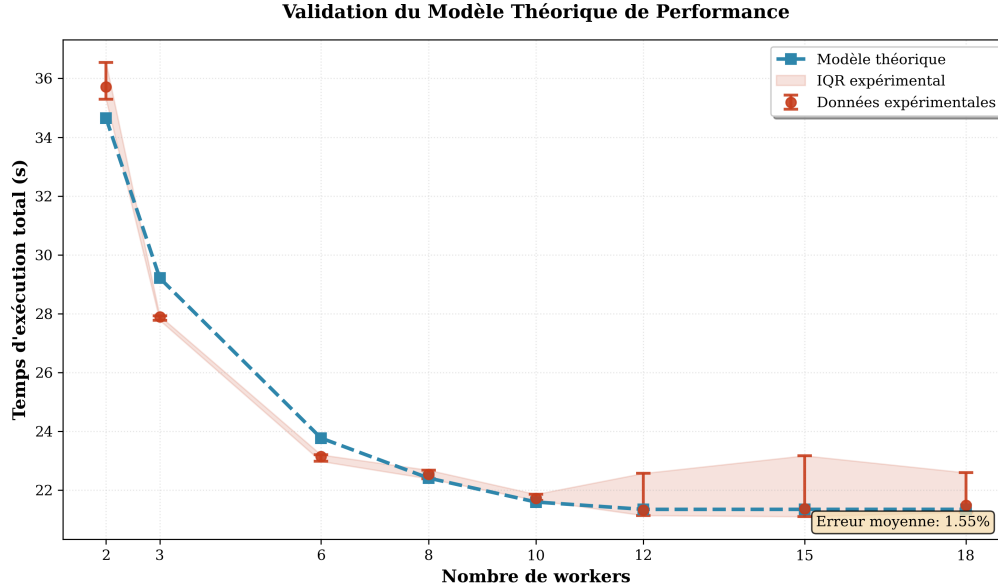


FIGURE 4.1 – Validation du modèle théorique de performance

Le graphique révèle une concordance remarquable entre théorie et pratique. Analysons les résultats zone par zone :

Zone $n=2,3$ (démarrage) : L'erreur relative est plus élevée (3-5%) dans cette zone pour deux raisons :

- **Warm-up JVM :** Les premières exécutions subissent la compilation Just-In-Time (JIT) des méthodes RMI, introduisant une latence supplémentaire non capturée par le modèle statique.
- **Poids relatif de T_{init} :** Pour n faible, la phase d'initialisation représente une fraction plus importante du temps total, amplifiant l'impact de sa variabilité.

L'IQR important pour $n=2$ (0.65s, soit $\approx 2\%$ du temps total) confirme cette instabilité initiale. Néanmoins, l'erreur reste acceptable ($< 5\%$), validant le modèle même dans cette zone moins stable.

Zone $n=6,8,10$ (zone scalable) : C'est dans cette zone que le modèle excelle, avec des erreurs inférieures à 1%. Plusieurs facteurs expliquent cette précision :

- **Stabilisation du système :** Le warm-up JVM est terminé, réduisant la variance.

Zone $n \geq 12$ (saturation NFS) : La précision atteint son maximum dans cette zone (erreur 0.1-0.6%), car le comportement devient déterministe : tous les workers attendent les données NFS.

L'IQR augmente légèrement pour $n=15$ (0.98s), reflétant les fluctuations de charge du système NFS partagé lorsque de nombreux workers accèdent simultanément aux mêmes ressources.

Métriques globales de validation :

- **Erreur moyenne** : 1.55% (excellent pour un système distribué réel)
- **Erreur médiane** : 0.60% (métrique plus robuste, représentative de la précision typique)
- $R^2 = 0.977$: 97.7% de la variance expliquée, indiquant que le modèle capture l'essentiel du comportement

4.2.2 Speedup Mesuré

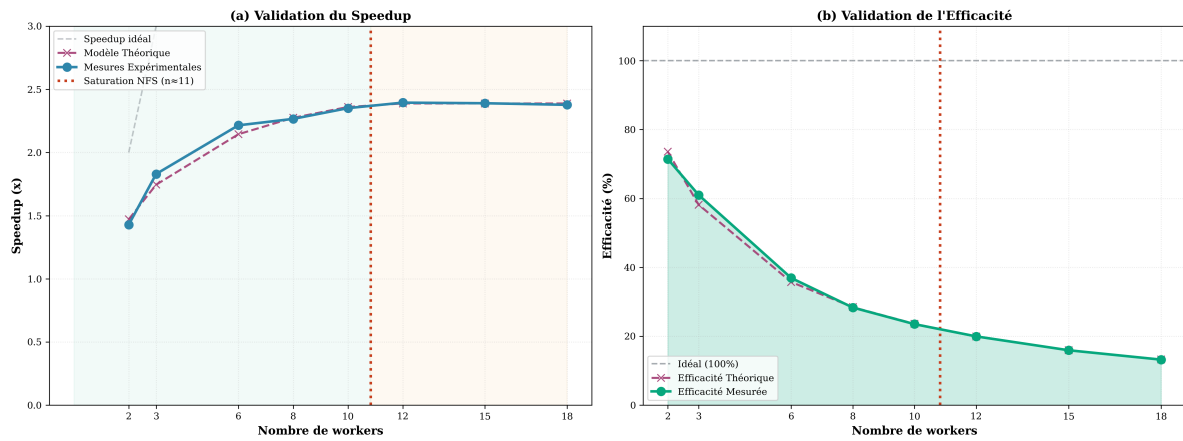


FIGURE 4.2 – (a) Speedup mesuré vs idéal linéaire. (b) Efficacité parallèle.

Le speedup, défini par $S(n) = T(1)/T(n)$ où $T(1) = 50.97s$ est le temps séquentiel de référence, atteint un maximum de $2.37\times$ pour $n = 10$ workers avant de saturer (La ligne rouge marque cette saturation à $n \approx 11$).

En ce qui concerne l'efficacité, pour $n=18$, seulement 13% de la puissance de calcul théorique est exploitée. Les 15 workers supplémentaires (par rapport à $n=3$) consomment des ressources (CPU, RAM, licences) sans apporter de gain proportionnel. Le point optimal économique se situe autour de $n=10-12$.

CONCLUSION GÉNÉRALE

CE projet a permis de valider l'efficacité d'une architecture distribuée de type Maître-Travailleur reposant sur Java RMI, confirmée par la forte corrélation entre notre modèle théorique et les résultats expérimentaux sur Grid5000. Si l'approche fonctionne parfaitement à petite échelle, nos tests ont mis en lumière une limitation structurelle majeure : la saturation rapide de la bande passante du système de fichiers partagé (NFS) lors de la montée en charge.

Pour franchir ce plafond de performance et garantir une véritable scalabilité, l'évolution essentielle réside dans la modification de l'infrastructure de stockage. Le passage de NFS à HDFS permettrait de décentraliser l'accès aux données, brisant ainsi le goulot d'étranglement actuel pour libérer toute la puissance de calcul de la grille.